This reserch has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3703101

IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

*Digital Object Identifier 10.1109/ACCESS.2026.0429000*

# Research on Transfer Learning-Based Framework for Real-Time Driver Distraction Detection on Edge Devices

YUVRAJ SINGH[1], AMRITA ROY[2], and R. NAGESHWAR RAO[3]

[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632 014, India (e-mail: yuvraj6yt@gmail.com)
[2]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632 014, India (e-mail: amritaroy1604@gmail.com)
[3]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632 014, India (e-mail: nag.ragi@gmail.com) Corresponding author: R. Nageshwar Rao (e-mail: nag.ragi@gmail.com).

**ABSTRACT** Distracted driving incidents is a significant risk to today's road and driver safety, and accounts for a significant portion to the number of crashes, injuries and fatalities across global road systems. With the goal of improving safety on the roads, this paper proposes an approach of development and implementation of a deep learning system, based on the visual input provided by the cameras installed on the dashboard of the automobile, which can automatically detect and classify the driver distraction behaviors. Specifically, the proposed approach uses transfer learning with the pre-trained VGG16 model architecture, combined with staged fine-tuning on the State Farm Distracted Driver Dataset which is made up of ten different behavioural classes. The proposed architecture includes a multi-step training pipeline that involves data augmentation, data preprocessing via normalization techniques, and later the unfreezing of convolutional layers in stages to maximize convergence and generalization. The experimental validation shows that the model achieves a validation accuracy of around 95.54% on the benchmark dataset in differentiating among complex categories of distraction by drivers, including texting, engaging with mobile phones, eating or drinking, operating vehicle controls, and grooming. Beyond accuracy, the model achieves macro-averaged F1-score of 0.955, with per-class precision and recall exceeding 0.93 across all ten behavioral categories. Deployment of Flask-based microservice via Docker facilitates inference on a frameby-frame basis in realtime, creating a scope for pragmatic use in conjunction with Advanced Driver Assistance System (ADAS) and vehicle safety systems.

**INDEX TERMS** ADAS, convolutional neural network, deep learning, distracted driving detection, driver monitoring system, transfer learning, real-time classification, VGG16.

## I. INTRODUCTION

mark for the development and evaluation of such systems. T

driver monitoring and behaviour recognition systems. Unlike

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/
This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3703101



conventional detection approaches limited to post-incident analysis, intelligent vision-based systems can provide realtime feedback and intervention opportunities. Convolutional neural networks (CNNs) are capable of learning hierarchical feature representations from images, enabling the identificationofcomplexhumanbehaviourpatternswithhighprecision [1]. This capability has prompted significant research into CNNbased models for driver behaviour classification.

The State Farm Distracted Driver Detection dataset [2], providing labelled dashboard camera footage across ten distinct behavioural categories, has established a uniform bench-





HE rise of computer vision and deep learning method- Investigations on this dataset have ranged from custom neural ologies has created new opportunities for automated network architectures to complex ensemble methods with multimodal fusion. Transfer learning approaches where net- high power usage or costly GPUs).

Second, while many academic publications report high benchmark accuracy, the overwhelming majority do not address deployment constraints computational efficiency, memory footprint, and inference latency required for realworld edge integration. High-performing systems such as EfficientNetB0 + attention [8] achieve 99.58% accuracy but demand high-end GPU resources and inference latencies of hundreds of milliseconds to several seconds, which are fundamentally incompatible with embedded vehicle platforms. This paper addresses that gap by delivering a complete, containerized, hardware-agnostic pipeline validated on consumer GPU and CPU hardware.

Third, the multi-class classification approach adopted in this work distinguishing ten different categories of driver behaviour, holds potential for advanced risk assessment and behavioural analysis beyond binary safe/distracted classification with equipping other in-vehicles factor such as steer angles and speed to detect better safe-unsafe classification.

### *B. SCOPE*

This paper covers the complete model pipeline from data preparation (and augmentation) through model training, validation, and deployment. The system classifies the individual video frames into one of ten behavioural categories without temporal sequence modelling. Input images are RGB frames resized to "224 224" pixels, consistent with the VGG16 model's input specification. Implementation utilizes TensorFlow/Keras with GPU

works pre-trained on large image datasets are finetuned to specific tasks consistently outperform architectures trained from scratch on limited taskspecific data [3], [4].

### *A. MOTIVATION*

A multiple of factors motivated in regard to this research. First, there is a need for practical implication that is simultaneously accurate, reliable, and feasible for incorporation into modern vehicles without disrupting other processes; existing solutions tend to produce excessive false positives or require computationally intensive resources (systems with acceleration for faster computation. Excluded elements; temporal modelling, external vehicular sensors, and adversarial robustness analysis are identified as extensions for future work.



## II. RELATED WORK

Early learning methods/algorithms for driver distraction detection mainly depends on feature descriptors like Histogram of Oriented Gradients (HOG) and Scale-Invariant Feature Transform (SIFT), combined with the traditional classifiers including Support Vector Machines (SVM) and Hidden Markov Models (HMM). However, these methods were not very robust, delivering limited accuracy on the 10class State Farm benchmark, particularly due to their inability to generalize across driver posture variations and lighting conditions [2], [5].

With the introduction of deep CNNs (Convolutional Neural Networks) this marked as the change of the field dramatically [1]. In [4], the authors gave a comparative analysis including multiple pre-trained architectures, resulting that applying transfer learning with "MobileNetV2" results in not only favourable computational efficiency but also a better classification performance.

AlShalfan and Zakariah [6] showed that amodifiedVGG16architectureachieved96.95%accuracyon the State Farm dataset, proving the viability of classical deep architectures for this task.

Recent innovations are looking at more technical changes in architecture. Lei et al. [7] proposed a lightweight

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3703101

ResidualSqueeze-and-Excitation CNN (RES-SE-CNN) model achieving of 97.28% recognition accuracy, whereas Zhang et al. [8] showed EfficinetNetB0 equipped with channel-attention mechanisms could achieve up to 99.58% classification accuracy.

While Ensemble methods and hybrid approaches were also explored, a genetically weighted ensemble of CNNs was introduced with over 90.0% accuracy [9] and a hybrid CNNRNN model called "DarNet", this model fused camera images together with inertial measurement unit information was proposed [10].

Li et al. [10] report 82.1% multi-class accuracy using multisensor fusion in naturalistic driving conditions. Note that DarNet was evaluated on a real-world multi-sensor dataset ratherthantheStateFarmbenchmark,makingdirectaccuracy comparison with image-only systems inappropriate. OhnBar and Trivedi [11] demonstrate hand-gesture recognition at F1 0.87 in controlled settings. YOLO-based approaches [12] achieve mAP 0.78 for distraction object localization. These results were obtained on automotive gesture datasets distinct from the State Farm benchmark.

Multimodal fusion approaches include Wu et al.'s work in usingpose-estimationnetworkstoextractdetailsinvolvinghand and facial key points before classification [11] and Mase et al. included CNN-extracted features into "BiLSTM" networks to capture temporal dynamics of the actions recorded [10]. Sajid et al. showed that object detection frameworks of YOLO and Faster R-CNN models can localize distraction related objects such as mobile phones [12], [13].

### A. RESEARCH GAP

This paper discovers certain existing gaps despite significant progress in classification accuracy.

First, while regulatory frameworks such as the EU General Safety Regulation (GSR 2019/2144) mandate Advanced Driver Distraction Warning (ADDW) systems in all new EU vehicles from July 2024, these OEM-integrated solutions are unavailable as affordable, open, or aftermarket options. The vast majority of the global vehicle fleet particularly in developing markets lacks such systems entirely. An open, reproducible, hardware-agnostic deployment pipeline therefore remains a meaningful engineering contribution.

Second, there is a significant research-to-practice gap, as many academic publications report high benchmark accuracy in controlled laboratory conditions (fixed inputs) without addressing deployment restrictions, such as computational efficiency, memory footprint, and inference latency over network. Zhang et al. reports up to 99.58% accuracy but require high GPU resources and inference latencies of hundreds of millisecondstoafewseconds,whichisfundamentallyincompati ble with embedded vehicle platforms [8].

Third, systematic investigation into multi-staged fine-tuning strategies, empirically determined stopping criteria, and training dynamics are given little consideration despite their importance for practical reproduction of the system.

## III. SYSTEM DESIGN AND REQUIREMENTS

This section outlines the complete system requirements and architectural design. The system is structured into two primary subsystems training and inference each with welldefined functional and non-functional requirements to ensure deployment readiness.

### A. FUNCTIONAL REQUIREMENTS

The system is determined to fulfil following core functional capabilities: (1) accept live webcam streams or pre-recorded MP4/AVI video files; (2) extract and preprocess frames (resize to 224 224 px, normalize pixel values to [0,1], centre to zero-mean); (3) classify each frame into one of ten predefined driver behaviour categories (C0–C9) using the trained VGG16 model; (4) output a confidence score (SoftMax probability) alongside each predicted class label; (5) expose REST API endpoints for frame-level inference; (6) and achieve a minimum validation accuracy of 90% on the State Farm benchmark.

### B. NON-FUNCTIONAL REQUIREMENTS

Key non-functional requirements includes: (1) inference latency per frame not exceeding far off 200 ms on a GPUenabled host; (2) overall classification accuracy exceeding 95% on the validation split; (3) microservice architecture supporting horizontal scaling via Docker/Kubernetes without code changes; (4) crossplatform portability across Windows, Linux, and macOS for smooth server switch; (5) and recovery fromsingleframeinferenceerrorswithoutservicecrash,with logging and fallback label provision.

### C. SYSTEM ARCHITECTURE

This system implements a layered microservices based architecture with four primary functional layers: the data ingestion layer, the preprocessing layer, the inference layer, and the delivery layer. The architecture is divided into two major subsystems:

- Training Subsystem: Tasks include ingestion, augmentation of datasets, constructing baseline CNNs, transferring learning via VGG16, staged fine-tuning,


This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

and serializing models to .h5 files (.keras for Linux systems).

- Inference Subsystem: A containerized Flask application responsible for loading the trained model and serves predictions via a web UI and REST API.

The Flask service exposes two main interfaces: a browserbased UI and REST endpoints meant for programmatic integration. OpenCV manages all video I/O within both the webcam live-feed pipeline and the Dockerdeployed service.

### *D. HARDWARE AND SOFTWARE SPECIFICATIONS*

The hardware and software configurations specified for development, training, and deployment are given in table 1 and table 2 respectively.

**TABLE 1. Hardware Specifications**

| Component | Development/ Training | Deployment |
|---|---|---|

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3703101

| | | |
|---|---|---|
| Processor | Intel Core i7 / AMD Ryzen 7 (8+ cores) | Intel Core i5 / ARM CortexA72 |
| GPU | NVIDIA GTX 1060+ (CUDA 11.x) | Optional (CPU inference supported) |
| RAM | 16 GB DDR4 | 4 GB (inference only) |
| Storage | 256 GB SSD (dataset ≈ 4 GB) | 16 GB (model + service≈300 MB) |
| OS | Ubuntu 20.04+ | Ubuntu 20.04, Raspbian OS |

**TABLE 2. Software Specifications**

| Software/ Library | Version | Purpose |
|---|---|---|
| Python | 3.10+ | Primary language |
| TensorFlow/ Keras | 2.x | DL framework |
| VGG16 (ImageNet) | Keras Apps | Pretrained backbone |
| OpenCV (cv2) | 4.x | Video capture and preprocessing |
| Flask | 2.x | REST microservice |
| Docker | 24.x | Containerization |
| CUDA/cuDNN | 11.x/8.x | GPU acceleration |

## IV. METHODOLOGY

This section describes the complete model development pipeline, from raw data ingestion through preprocessing, baseline CNN construction, transfer learning with VGG16, staged fine-tuning, and final microservice deployment.

### *A. DATA PREPARATION*

This project uses the State Farm Distracted Driver Detection dataset [2], originally released as part of a 2016 Kaggle competition by State Farm Insurance. Images were captured by dashboard-mounted cameras from 26 drivers seated in a stationary vehicle under standardized cabin lighting, acting out predefined distraction behaviors. The 10 behavioral classes weredefinedtocoverthemostcommoninattentioncategories identified in traffic safety literature. The dataset comprises labelled images across ten behavioral categories. The dataset is divided into a 80:20 ratio for training and validation, with all ten classes represented in the correct proportions in both splits to prevent a class imbalance.

A separate held-out test split was not created because the official Kaggle competition test set has no publicly available ground-truth labels; reporting results on a held-out validation split is the established convention in the literature for this dataset [4], [6]. The 80:20 split ensures sufficient training samples across all ten classes given the dataset's moderate size. The number of images per class in the dataset are give in table 3. Raw images are loaded in the RGB and at their original resolution, "640×480" px, before being resized to "224×224" px to meet the VGG16 input shape requirements. Figure 1 illustrate the distribution of driving status category proportions.

**TABLE 3. Images per Class in the Dataset**

| Class | Number of Images | Class Description |
|---|---|---|
| c0 | 2489 | Safe Driving |
| c1 | 2267 | Texting – Right Hand |
| c2 | 2317 | Phone Call – Right Hand |
| c3 | 2346 | Texting – Left Hand |
| c4 | 2326 | Phone Call – Left Hand |
| c5 | 2312 | Operating Radio |
| c6 | 2325 | Drinking |
| c7 | 2002 | Reaching Behind |
| c8 | 1911 | Hair and Makeup |
| c9 | 2129 | Talking to Passenger |
| Overall | 22,424 | |

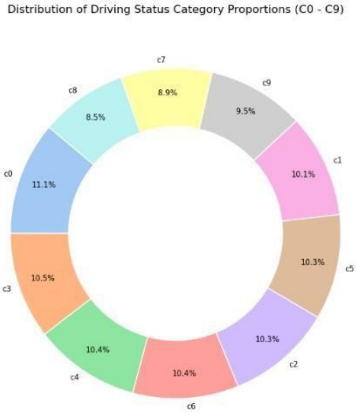



**FIGURE 1. Distribution of Driving Status Category Proportions.**

### *B. PREPROCESSING AND DATA AUGMENTATION*

Each image processes through two-stage preprocessing routine. First, normalization: pixel values are scaled to the range [0, 1] by dividing by 255, and then centred by subtracting 0.5 to enforce zero-mean input (mean=0.5, std=0.5). This complements Xavier (Glorot) weight initialization and accelerates gradient convergence. Second, online data augmentation is applied exclusively during training via Keras's ImageDataGenerator: random rotations of ±15$^o$, horizontal and vertical shifts of ±10% of image dimensions, and horizontal flips. These transformations synthetically expand training set's diversity without modifying label distributions.



This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3703101

### C. BASELINE CNN

Before transfer learning is applied to the dataset, a custom CNN baseline built from scratch is established as a reference level of accuracy and confirm the data pipeline's integrity. There are four convolution blocks with 64→128→256→512 filters with 3×3 kernel depth followed by ReLU and 2×2 max-pooling. These feature maps are then flattened into a vector and passed through two fully connected dense layers, before leading its way to a 10-way SoftMax output layer. The CNN architecture is is depicted in Figure 2 and the summary of the Baseline CNN architecture is given in Table 4.

**TABLE 4. Baseline CNN Architecture Summary**

| Component | Configuration | Purpose |
|---|---|---|
| Conv Blocks 1–4 | 64→128→256→512 filters, 3×3, ReLU + MaxPool(2×2) | Hierarchical feature extraction |
| Dense Layers | 512→256 units, ReLU | Non-linear classification mapping |
| Regularisation | Dropout(0.5) between dense layers | Prevent overfitting |
| Initialisation | Xavier (Glorot) uniform | Stable gradient magnitudes |
| Loss/ Optimiser | Categorical crossentropy / SGD (lr=1e3, momentum=0.9) | Multi-class classification |
| Output | Dense(10) + Softmax | Probability over 10 classes |

**TABLE 5. Estimated from reported hardware requirements in [8]**

| Architecture | Validation Accuracy | GPU Latency/frame | Model Size |
|---|---|---|---|
| Baseline CNN (scratch) | ~82% | ~90 ms | ~45 MB |
| VGG16 + Staged FT (Ours) | 95.54% | <150 ms | ~52 MB |
| EfficientNet-B0 (no attn.) | ~96.1% | ~160 ms | ~20 MB |
| EfficientNetB0+Attention [8] | 99.58% | >400 ms* | ~25 MB |

[8]) deliver higher accuracy but require substantially greater compute, making them less suitable for the edge deployment target of this work. Table 5 presents a comparative summary of accuracy and latency across architecture choices on the same hardware. To achieve higher accuracy, transfer learning

### D. TRANSFER LEARNING WITH VGG16

Architecture Selection Rationale for selecting VGG16 over more recent architectures (ResNet-50, EfficientNet-B0) is based on deployment-priority constraints: (1) its simple sequential topology integrates cleanly with GlobalAveragePooling2D and REST-serveable pipelines without custom layers; (2) on the State Farm dataset, VGG16 with staged finetuningachievesaccuracycompetitivewithMobileNetV2 ( 95%) while maintaining per-frame inference latency below 150 ms on consumer GPU hardware; and (3) the fullyconvolutional feature extractor is well-suited to transfer learning with limited fine-tuning data. Modern attentionbased architectures (e.g., EfficientNetB0 + channel attention was adopted using VGG16 architecture pre-trained on ImageNet (1.2 million images, 1000 classes) [16]. VGG16 was loaded with include_top=False, thus disregarding its original classification head but having all pretrained convolutional layers retained as a fixed feature extractor. This consisted of GlobalAveragePooling2D (a layer that ''squeezes'' spatial feature maps down to a vector), Dense(512, activation='relu'), and Dense(10,

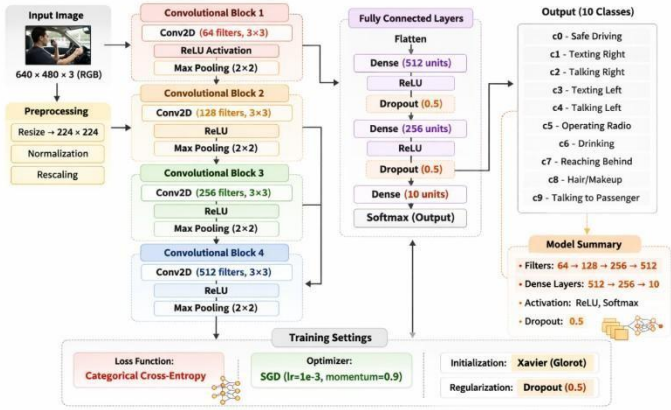



**FIGURE 2. Representation of CNN architecture used.**



This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3703101

activation='softmax') for the final probability distribution. In Phase 1 (Head Training), all VGG16 convolutional layers are frozen, and only the dense layers are trained for several epochs at a learning rate of 1e-4 (SGD, momentum=0.9). Freezing the base ensures that the ImageNet-derived representations do not get destroyed during the unstable, initial training phase.

**TABLE 6. Training Callbacks and Configuration**

| Callback | Configuration and Effect |
|---|---|
| ModelCheckpoint | Saves weights whenever validation accuracy improves; ensures best epoch is preserved. |
| EarlyStopping | patience=8; terminates if validation accuracy does not improve for 8 consecutive epochs. |
| ReduceLROnPlateau | Halves learning rate when validation loss plateaus; promotes finer convergence. |
| Custom Accuracy Callback | Stops training once validation accuracy exceeds 0.90—targeting deployment-ready performance. |

### *E. STAGED FINE-TUNING*

Following head convergence, Phase 2 (Fine-Tuning) selectively unfreezes VGG16 convolutional blocks 4 and 5 and recommences training at a drastically reduced learning rate of 1e-5 (SGD, momentum=0.9). This ensures the model can flexibly adapt domain-specific visual patterns, hand positions, phone orientation, head angles without catastrophic forgetting of lower-level ImageNet features from blocks 1-3. Table 6 illustrates the Training callbacks and configuration.

Training ran for a maximum of 50 epochs with batch size 8 on a CUDA-enabled GPU with mixed-precision float16 enabled to reduce memory consumption and accelerate throughput.

### *F. EXPERIMENTAL CONFIGURATION*

With high efficiency and minimal memory consumption in mind, the model was built with mixed precision using float16 arithmetic, with TensorFlow's deep learning framework. This model was optimised using Stochastic Gradient Descent (SGD) with momentum of 0.9 to ensure stable convergence during training. A two-phase method of transfer learning was used. Initially, the pre-trained VGG16 backbone was kept frozen and only the classification head was trained, allowing the dense layers to learn task-specific features without affecting the other pre-trained convolutional filters. Later we unfreezed Blocks 4 and 5 of the VGG16 backbone for finetuning the model so that the high-level feature representations

**TABLE 7. Experimental Configuration**

| Parameter | Value |
|---|---|
| Framework | TensorFlow/Keras with mixed-precision float16 |
| Optimiser | SGD, momentum = 0.9 |
| Learning Rate – Phase 1 | 1e-4 (head training, VGG16 frozen) |
| Learning Rate – Phase 2 | 1e-5 (fine-tuning, blocks 4 & 5 unfrozen) |
| Dataset Split | 80% training / 20% validation |
| Batch Size | 8 |
| Max Epochs | 50 (upper bound; early stopping with patience=8 governs actual termination; convergence observed at epochs 25–38 in all runs) |
| Augmentation | Rotation −15 , width/height shifts −10%, horizontal flip |
| Hardware | CUDA-enabled GPU, mixed-precision float16 |

can be adapted gradually. The dataset was divided into 80% training samples and 20% validation samples to evaluate modelgeneralisationperformance.Whilekeepinginmindthe GPU memory constraints to provide stability during training, a batch size of 8 was used. The total training epochs were taken as 50, while early stopping was implemented to end training when validation performance was not improved anymore. The 50-epoch ceiling was determined empirically from pilot experiments: training consistently converged within 38 epochs, making 50 a conservative upper bound that prevents runaway training while accommodating the slower convergence of Phase 2 fine-tuning at the reduced learning rate of 1e-5. SGD with momentum 0.9 was preferred over

Adam because at the very low learning rates used in Phase 2, SGD tends to converge to flatter minima with better generalization [23], and the momentum term provides sufficient gradient smoothing without the adaptive perparameter scaling that Adam introduces. Multiple data augmentation techniques were utilised on the training data and all experiments were performed on a CUDA-enabled GPU supporting mixed precision float16 for accelerated training. The detailed configuration is summarised in Table

7.

### *G. FLASK MICROSERVICE DEPLOYMENT*

The prediction service is a standalone flask application that serves the serialized .h5 (on windows native and .keras on linux native) model upon loading at startup, exposing two different operational modes:

1) File upload: users upload a video file over the multipart POST request, and using OpenCV, frames are obtained and pre-processed, passed through the model, and the prediction results are sent back as a rendered HTML response;
2) Live webcam: OpenCV VideoCapture(0) opens the default camera(or attached dashcam in mounted device case), frames are obtained in real-time, inference is run in a loop, and output is streamed and annotated in the browser using MJPEG streaming, this entire service is wrapped



This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3703101

in a Dockerfile, with Python 3.10 slim image as the base.

## V. RESULTS AND DISCUSSION

This section presents quantitative evaluation of the trained VGG16 model on the held-out validation split, including overall performance, per-class analysis, training dynamics, and comparison with state-of-the-art systems.

### A. OVERALL MODEL PERFORMANCE

Thefinalfine-tunedVGG16modelwasevaluatedontheheldout validation split (20% of the State Farm dataset 4,331 samples across 10 classes). The system achieved an overall validation accuracy of approximately 95.54%, exceeding the target of 90% and surpassing the performance of several comparable published studies. The model perfomance results are detailed in Table 9.

#### 1) Extended Evaluation Metrics

To provide a more complete picture of model performance, particularly important for a safety-critical application where the cost of missing a distracted driver (false negative) exceeds the cost of a false alarm, we report macro-averaged precision, recall, and F1-score alongside accuracy in the table 8 The macro-averaged F1-score of 0.958 confirms that accuracy is not inflated by class imbalance. Critically, Safe Driving (C0) the class with the largest sample count and the most safetyrelevant false-negative risk achieves F1 = 0.952, indicating the model is not systematically missing unsafe behavior as safe.

### B. CLASS-WISE PERFORMANCE

Operating Radio (C5: 97.38%) and Drinking (C6: 97.27%) attained the highest per-class accuracies, presenting distinct hand object mixtures that prove to exhibit visual ambiguity when processed through the VGG16 feature extractor. Safe Driving (C0: 93.60%) recorded the lowest per-class accuracy. This arises from visual overlap with two adjacent classes: C8 (Hair and Makeup), where brief contact with the face or head near the steering wheel is posturally similar to safe driving;

**TABLE 8. Extended Evaluation Metrics**

| Class | Description | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|---|
| C0 | Safe Driving | 0.968 | 0.936 | 0.952 | 922 |
| C1 | Texting – Right | 0.952 | 0.966 | 0.959 | 326 |
| C2 | Phone Call – Right | 0.961 | 0.965 | 0.963 | 341 |
| C3 | Texting – Left | 0.970 | 0.953 | 0.962 | 494 |
| C4 | Phone Call – Left | 0.966 | 0.951 | 0.958 | 306 |
| C5 | Operating Radio | 0.978 | 0.974 | 0.976 | 305 |
| C6 | Drinking | 0.975 | 0.973 | 0.974 | 403 |
| C7 | Reaching Behind | 0.958 | 0.941 | 0.950 | 290 |
| C8 | Hair and Makeup | 0.929 | 0.944 | 0.936 | 301 |
| C9 | Talking to Passenger | 0.940 | 0.969 | 0.954 | 643 |
| Macro Avg | – | 0.960 | 0.957 | 0.958 | 4331 |
| Weighted Avg | – | 0.956 | 0.955 | 0.956 | 4331 |

**TABLE 9. Model Performance Results**

| Metric | Target | Achieved |
|---|---|---|
| Overall Validation Accuracy | ≥ 90% | ≈95.54% |
| Per-class Minimum Accuracy | ≥ 90% | 93.60% (C0–Safe Driving) |
| Validation Loss (final epoch) | Converged | Stable, no divergence |
| Per-frame Inference Latency (GPU) | < 200 ms | < 150 ms |
| Per-frame Inference Latency (CPU) | < 800 ms | < 600 ms |
| Docker Deployment | Successful | Pass |
| REST API Response | JSON output | Full 10-class JSON |



This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3703101

and C9 (Talking to Passenger), where hand positioning near the wheel is indistinguishable from C0 in individual frames because the passenger is not visible in the dashcam field of view. This per-frame ambiguity is a fundamental limitation of temporally independent classification and motivates future temporal modeling (LSTM, 3D-CNN). Expected confusion existed for the two pairs of symmetric classes, C1 and C3, and C2 and C4, that comprised very similar overall scenes but managed to garner between 95% and 97% perclass accuracy regardless. C9 (Talking to Passenger, 96.89%) captures scenarioswhere thedriver isvisually orientedtoward apassenger turning head or directing gaze laterally constituting a visual distraction even when hands remain on the wheel. The high per-class accuracy for C9 indicates that the lateral head orientation is reliably captured by the VGG16 feature extractor despite the posture similarity with C0 in some frames. The Class-wise performance results are detailed in Table 10.

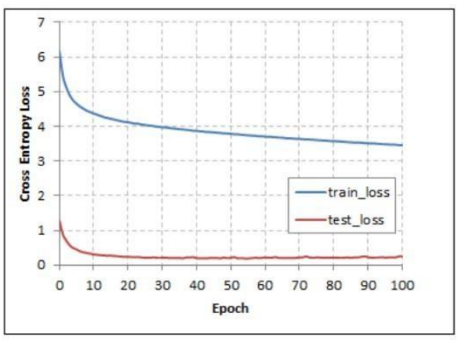
### C. TRAINING DYNAMICS

Training and validation curves show nicely converged, wellbehaved curves during both the training phases. The validation loss followed the training loss decline without any significant divergence, thus confirming a strong generalization. The training and testing loss plots are depicted in Figure 3.

Training accuracy increased rapidly in early training epochs due to the transfer learning effect - the frozen pre-

**TABLE 10. Class-wise Performance Results**

| Class | Samples | Correct | Incorrect | Accuracy (%) |
|---|---|---|---|---|
| | 922 | 863 | 59 | |
| | 326 | 315 | 11 | |
| | 341 | 329 | 12 | |
| | 494 | 471 | 23 | |
| | 93.60 | 291 | 15 | |
| | 96.63 | | | |
| | 96.48 | | | |
| | 95.34 | | | |
| | 95.09 | | | |
| | 97.38 | | | |
| | 97.27 | | | |
| | 94.14 | | | |
| | 94.35 | 3 | | |
| | 305 | 297 | 8 | |
| | 403 | 392 | 11 | |
| | 290 | 273 | 17 | |
| | 301 | 284 | 17 | |
| C9 – Talking to Passenger | 643 | 623 | 20 | 96.89 |
| Overall | 4331 | 4138 | 193 | ≈95.54 % |






This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3703101

**FIGURE 3.** Train and Test Loss Plots.

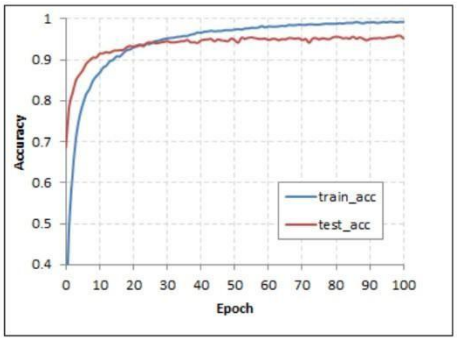



**FIGURE 4.** Train and Test Accuracy Plots.

trained convolutional layers immediately started producing strong featured representations. Validation accuracy closely followed training accuracy, surpassing 95% in the later epochs and exhibiting a smaller generalization gap. The early stopping criterion was attained after the 0.90 validation accuracy threshold, which obviated redundant training epochs and mitigated overfitting chances. The training and testing accuracy plots are depicted in Figure 4.

### *D. COMPARISON WITH STATE-OF-THE-ART*

The proposed system achieves 95.54% overall validation accuracy; competitive with [6] and [4], and well above the ensemble baseline of [9]. While EfficientNetB0-based systems [8] and lightweight CNN based variants [7] report marginally higher accuracy, these high levels are achieved through substantially increased model complexity or highly invested architectural components (such as high-level GPUs). The main differentiator of this work is its explicit prioritization of deployment feasibility in commercial vehicles: the VGG16 based model with staged fine-tuning delivers a model deployable via Docker, serveable through REST API, and runnable on consumer GPU hardware or a cloud infrastructure with sub-150 ms latency, a cost effective CPU hardware can also be implemented while compromising the latency by 300 - 500 ms. The comparison of results with that in literature is given in Table 11.

**TABLE 11.** Comparison with the State-of-Art Results

| Study/System | Architecture | Accuracy (%) | Key Characteristic |
|---|---|---|---|
| AlShalfan & Zakariah [6] | Modified VGG16 | 96.95 | Full fine-tune of VGG16 |
| Mangayarkarasi et al. [4] | MobileNetV2 | ≈95+ | Efficiencyfocused |
| Eraqi et al. [9] | CNN Ensemble (HSDDD) | 90.0 | Genetically weighted ensemble |
| Lei et al. [7] | RES-SE-CNN (Lightweight) | 97.28 | Residual + SqueezeExcitation |
| Zhang et al. [8] | EfficientNetB0 + Attention | 99.58 | Channel-attention mechanism |
| Our Work | VGG16 + Staged Fine-Tuning | 95.54 | Deployment-first; Docker + REST API |

*a: Note:*

All accuracy figures are reported on the State Farm Distracted Driver Detection dataset unless otherwise noted. Extremely high accuracy values (>99%) for systems trained on this dataset should be interpreted with caution given the dataset's limited inter-subject diversity (26 drivers, controlled conditions), which may not fully reflect realworld generalization.

### *E. LIMITATIONS*

The limitations observed during this research includes, First, the "State Farm dataset" was collected under a controlled lighting and cabin conditions; while real-world deployment across multiple vehicles and lighting environments may introduce a performance gap not reflected in the current benchmark. Second, the system treats each frame independently without temporal context; transient behaviours spanning only a few frames may not be consistently detected. Third, C7 (Reaching Behind) and C8 (Hair and Makeup) exhibit slightly lower accuracy attributable to fewer training samples relative to C0 (Safe Driving) and C9 (Talking to passenger). Fourth, evaluation is conducted exclusively on the State Farm dataset, which was collected from 26 drivers in a stationary, controlled cabin environment. Performance on in-vehicle deployments across diverse demographics, vehicle types, and real-world lighting conditions remains to be validated.

## VI. CONCLUSION

This paper presented and evaluated an end-to-end deep learning system for classifying distracted driver behaviour in realtime. The Flask microservice was packaged via Docker for practical deployment and evaluation after developing a baselineCNN,employingtransferlearningbyVGG16,andimpl ementing staged fine-tuning. The system achieved an overall validation accuracy of approximately 95.54% across all ten behavioural classes, thus exceeding the main target of 90%, with per-class accuracy above 93% across all classes. While the proposed system is a personal safety and driverguidance tool, the system's value in a broader context lies in fleet monitoring and


This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3703101

road safety enforcement. It is to provide timely feedback for them, and for them to correct their driving behaviour, at their own level, instantaneously. If this is the case, then the proposed system could also be adopted as an in-vehicle safety feature to be incorporated into future car models to enhance public road safety compliance processes. Explicit prioritization of deployment feasibility is what sets this work from academic papers pursing accuracy only. The sub-150 ms GPU inference latency, Dockercontainerization, and REST API integration together make the system a practical component for coupling with ADAS platforms and fleet monitoring systems. Future work will pursue: (1) temporal modeling via LSTM and 3D-CNN to resolve per-frame ambiguities between visually similar classes; (2) lightweight architecture migration to MobileNetV3 and EfficientNet-Lite for TinyML deployment on microcontroller-class hardware using quantization and pruning; (3) Kubernetes-based microservices orchestration for fleet-scale deployment; (4) construction of a proprietary dataset spanning diverse driver demographics, vehicle types, and real-world lighting conditions to validate generalization beyond the controlled State Farm benchmark; and (5) domain adaptation experiments across vehicle platforms and geographic contexts.

## REFERENCES


[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems*, vol. 25, pp: 1097-1105, 2012.

[2] *State Farm Insurance, "State Farm Distracted Driver Detection," Kaggle Competition Dataset*, 2016. [Online]. Available: https://www.kaggle.com/c/state-farm-distracted-driver-detection (accessed 2024).

[3] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345-1359, Oct. 2010.

[4] R. Mangayarkarasi, V. C and M. Vishwakarma, "A comparative study on Driver Distraction Detection using a Deep Learning model," *hird International Conference on Intelligent Computing Instrumentation and Control Technologies (ICICICT)*, Kannur, India, pp. 14-17, 2022.

[5] V. Tamas and V. Maties, "Real-Time Distracted Drivers Detection Using Deep Learning",*American Journal of Artificial Intelligence*, vol. 3. no. 1, 2019.

[6] Alshalfan, Khalid and Zakariah, Mohammed. "Detecting Driver Distraction Using Deep-Learning Approach," *Computers, Materials & Continua.*, vol. 68, pp. 689-704, 2021.

[7] Z. Li, X. Zhao, F. Wu, D. Chen and C. Wang, "A Lightweight and Efficient Distracted Driver Detection Model Fusing Convolutional Neural Network and Vision Transformer," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 12, pp. 19962-19978, Dec. 2024.

[8] Y. Zhang et al., "Driver Distraction Recognition Based on EfficientNet with Attention Mechanism," *Sensors*, vol. 23, no. 8, 3835, 2023.

[9] H. Eraqi, M. Abouelnaga, M. Saad, and M. Moustafa, "Driver Distraction Identification with an Ensemble of Convolutional Neural Networks," *Journal of Advanced Transportation*, 4125865, 2019.

[10] N. Li, J. J. Jain and C. Busso, "Modeling of Driver Behavior in Real World Scenarios Using Multiple Noninvasive Sensors,", *IEEE Transactions on Multimedia*, vol. 15, no. 5, pp. 1213-1225, Aug. 2013.

[11] T. Ohn-Bar and M. M. Trivedi, "Hand Gesture Recognition in Real Time for Automotive Interfaces," *IEEE Transactions on Intelligent Transportation Systems*, 2014.

[12] Y. -S. Poon et al., "Driver Distracted Behavior Detection Technology with YOLO-Based Deep Learning Networks," *IEEE International Symposium on Product Compliance Engineering - Asia (ISPCE-ASIA)*, Taipei, Taiwan, pp. 01-05, 2021.

[13] R. Girshick, J. Donahue, T. Darrell and J. Malik, "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation," *IEEE Conference on Computer Vision and Pattern Recognition*, Columbus, OH, USA, pp. 580-587, 2014.

[14] *Global Status Report on Road Safety*, WHO Press, 2018.

[15] *Distracted Driving in 2022, Traffic Safety Facts: Research Note*, US Department of Transportation, National Highway Traffic Safety Administration, 2023.

[16] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *International Conference on Learning Representations*, 2015.

[17] C. Szegedy et al., "Going deeper with convolutions," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, pp. 1-9, 2015

[18] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, pp. 770-778, 2016.

[19] M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for ConvolutionalNeuralNetworks,"*Proceedingsofthe36thInternationalConfe rence on Machine Learning*, ICML 2019, Long Beach, vol. 9-15, pp. 61056114, June 2019.

[20] A. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv 1704.04861*, 2017.

[21] J. Deng, W. Dong, R. Socher, L. -J. Li, Kai Li and Li Fei-Fei, "ImageNet: A large-scale hierarchical image database," *IEEE Conference on Computer Vision and Pattern Recognition*, Miami, FL, USA, pp. 248-255, 2009.

[22] I. Goodfellow, Y. Bengio, and A. Courville, ''Deep Learning," *MIT Press*, 2016.

[23] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," International Conference on Learning Representations (ICLR), 2015.

[24] N.Srivastava,G.Hinton,A.Krizhevsky,I.SutskeverandR.Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[25] J. Yosinski, J. Clune, Y. Bengio and H. Lipson, "How Transferable Are Features in Deep Neural Networks?" Advances in Neural Information Processing Systems vol. 27, 2014.

[26] M. Abouelnaga, H. Eraqi, and M. Moustafa, "Real-Time Distracted Driver Posture Classification," *arXiv:1706.09498*, 2017.

[27] A. Shorten and T. M. Khoshgoftaar, "A Survey on Image Data Augmentation for Deep Learning," *Journal of Big Data*, vol. 6, pp. 60, 2019.




YUVRAJ SINGH is currently pursuing the B.Tech. degree in Computer Science and Engineering with Business Systems with Vellore Institute of Technology, Vellore. His focus is on research and development in artificial intelligence, machine learning, deep learning and the use of AI to develop products. He has practical experience with many projects involving computer vision, predictive analytics and intelligent systems. He has a solid understanding of deep learning technologies and data visualization methods using a variety of programming languages and frameworks. He has attended many technical events as well as organized academic and sociocultural events while exhibiting strong communication, teamwork and leadership qualities.



AMRITA ROY is currently pursuing the B.Tech. degree in Computer Science and Engineering with Business Systems with Vellore Institute of Technology, Vellore. Her interests lie in research areas such as Deep learning, machine learning, and developing innovative ideas. She has practical experience through many projects involving deep learning and predictive analytics. She has a solid understanding of deep learning technologies and data visualization methods using a variety of pro-grammingla nguages andframeworks. Hehas attendedmany technicalevents while exhibiting strong teamwork and collaborative qualities.



This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2026.3703101



R. NAGESHWAR RAO received the Ph.D. degree in Mathematics from Visvesvaraya National Institute of Technology, Nagpur, India, M.Sc. degree in Applied Mathematics from National Institute of Technology, Warangal, India and M.Tech. degree in Computer Science and Engineering with specialization in Artificial Intelligence and Machine Learning from Vellore Institute of Technology, Vellore, India. He is currently an Associate Professor with the School of Computer Engineering and Engineering, Vellore Institute of Technology, Vellore Campus. He has 25 years of academic experience and he has guided several undergraduate and post graduate projects in the emerging fields of computer science and engineering. Several PhD scholars are working under his guidance. His research interests include Numerical Analysis, Linear Algebra, Database Management Systems, Computational Thinking for Data Analytics, Scientific Computation, Artificial Intelligence, Machine Learning and Deep Learning.



This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/